



Security Assessment Report



Safe Sentinel Oracle

September 2026

Prepared for Safe

Table of Contents

Project Summary	3
Project Scope	3
Project Overview	3
Protocol Overview	4
Assessment Methodology	5
Threat and Security Overview	6
System Model	6
Invariants and Trust Boundaries	6
Attack Surface	7
Findings Summary	8
Severity Matrix	8
Detailed Findings	9
Low Severity Issues	11
L-01 `commit` accepts zero hashes, allowing repeated commits that lock bonds and distort accounting....	11
L-02 Unbounded concurrent bonding lets a sponsor lock sentinel capital per unit of fee spent, leading to sentinel exhaustion.....	12
Informational Issues	13
I-01. Terminal arbitrated or timed out requests do not emit `OracleResult`, stalling off-chain consumers....	13
I-02. Missing validation for non-zero `GOVERNANCE_DELAY` can disable governance timelocks.....	14
I-03. Fee-reward division permanently locks undistributed remainders.....	15
I-04. Non-revealer slashes are taken at freeze time and are never reversed, contradicting the documented "everyone made whole" behaviour of the timeout paths.....	16
I-05. `daoFeeShare` is never logged upon request creation.....	17
I-06. Push-based settlement to sponsor allows freezing of requests and trapping of sentinel bonds.....	18
I-07. Tokens with decimal precision larger than 18 may cause overflows.....	19
Disclaimer	20
About Certora	20

Project Summary

Project Scope

Project Name	Repository Link	Initial Commit	Final Commit	Platform
Safe Sentinel Oracle	https://github.com/safe-research/safenet	49d7e398	1d89801	Solidity

Project Overview

This document describes the security review of **Safe Sentinel Oracle**. The work was undertaken from **Sept 7th, 2026** to **Sept 14th, 2026**.

The following contract list is included in our scope:

- contracts/src/SentinelOracle.sol
- contracts/src/libraries/BondConfig.sol
- contracts/src/libraries/DelayedAddress.sol
- contracts/src/libraries/DelayedUint96.sol
- contracts/src/libraries/SentinelMap.sol
- contracts/src/libraries/SentinelOracleCommitments.sol
- contracts/src/libraries/SentinelOracleRequests.sol

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

SentinelOracle is an ERC-20-bonded, commit-reveal oracle for assessing submitted requests. A designated proposer posts a request and funds its fee; registered sentinels commit and later reveal approve/deny votes while posting bonds.

Commitments keep votes private during the commit phase, while the reveal phase records the outcome and enables the contract to identify unrevealed or losing participants for slashing. If revealed votes support only one outcome, the request is finalized directly; conflicting approve and deny votes freeze the request and escalate it to the configured arbitrator.

The arbitrator can resolve the dispute, let it time out, or mark it out of scope, with each path determining sponsor refunds, sentinel bond claims and rewards, DAO fee allocation, and transfers to the protocol funds receiver. The contract is intended to be called by the Consensus contract when proposing a Safe transaction, while governance-delayed configuration controls its economic parameters and operational time windows.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

System Model

SentinelOracle is a bonded commit-reveal voting contract. A trusted proposer opens a request against a Safe transaction and a governance-specified set of sentinels stake a bond and vote on whether that transaction is safe. Following the commit window, the reveal window takes place to reveal the Sentinels' votes. The verdict of the votes is emitted as an OracleResult event for the validator set to act on. The contract escrows money, enforces the voting schedule, counts votes, and pays out rewards, proposal fee refunds, and slashed bonds. Disagreement, where at least a single vote is received for/against a proposal, is escalated to a single human arbitrator. The SentinelOracle contract is one Solidity file backed by seven libraries, each controlling a part of the contract's state.

Invariants and Trust Boundaries

The principal invariants reviewed were that every request pays out exactly what it took in ($\text{fee} + \text{committedCount} \times \text{bondTarget}$) across all four final outcomes; that each commit call pulls exactly one bond so the escrow always covers committedCount; that a sentinel commits at most once per request; that the commit and reveal windows never overlap, so no vote can be cast or changed after another is seen; that revealedCount equals $\text{approveSentinelCount} + \text{denySentinelCount}$ and never exceeds committedCount; that slashingMultiplier stays within $[1, \text{bondMultiplier}]$, which is what keeps bondReturn from underflowing and keeps slashed $\geq \text{feeAmount}$ during dispute resolution; and that each commitment is claimed at most once.

Sentinels are a trusted voting set, not a quorum. GOVERNANCE alone decides the inclusion of EOAs in the Sentinel set; registration is delayed by GOVERNANCE_DELAY while removal is immediate; sentinels post no stake beyond their per-request bonds; and the registry cannot be counted or enumerated on-chain. One sentinel revealing approve with no dissent resolves a request outright, so each registered sentinel must be treated as individually capable of producing an approval.

The arbitrator holds full authority over any frozen request. A single call either slashes the entire losing side, however large that side is, or releases everyone. `resolveDispute` has no deadline check and only times out if someone calls `timeoutArbitration` prior.

The PROPOSER check on `postRequest` authenticates the calling contract, not the individual party broadcasting the transaction. `Consensus.proposeTransaction` is permissionless and its `oracleData` argument is unconstrained, so any address can open unlimited distinct requests against the same Safe transaction, paying the fee as sponsor and possibly recovering it in the case of timeout or lack of sentinel activity.

Each sentinel is a single EOA, so all of its commits, reveals, finalizations and claims serialize through one nonce sequence regardless of how many concurrent requests are open. Therefore, a limited sentinel set will have limited throughput for approving or denying transactions, especially when the commit/reveal windows are relatively short (e.g. several blocks).

Attack Surface

The audit considered malicious or compromised proposers, sponsors, registered sentinels acting alone or in collusion, the arbitrator, governance, the protocol funds receiver, the fee token, sentinel operators and their engines, and the permissionless callers of `finalize`, `timeoutArbitration` and `claim`. Review areas included commit hiding and binding, packed-encoding ambiguity in the commit preimage, salt derivation and reuse across requests and sentinels, window overlap and reveal ordering, the checks that gate double commits, uint overflow, and governance-defined parameters.

Execution and economic analysis covered value conservation across all four final outcomes, the SafeCast boundaries and `uint96` and `uint128` widths on fee, bond and slash arithmetic, integer-division dust in the fee split, every path that refunds the sponsor, the cost of manufacturing a freeze or withholding a reveal, capital lockup and the bond-to-fee leverage ratio, and participation incentives at varying sentinel counts. Off-chain theorization revolved around the sentinel's single-EOA nonce serialization, deadline expiry due to inability to commit to proposals, and reorg lag measured against the commit and reveal windows.

Cross-component review included the EIP-712 derivation of the request ID in Consensus, IOracle conformance, Rust and Solidity parity on the commit hash and salt derivation, non-standard ERC-20 behaviour including fee-on-transfer, rebasing, and high-decimal tokens, and which oracle events the downstream validator client actually parses.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	–	–	–
Medium	–	–	–
Low	2	2	1
Informational	7	7	6
Total	9	9	7

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
L-01	<code>commit</code> accepts zero hashes, allowing repeated commits that lock bonds and distort accounting	Low	Fixed
L-02	Unbounded concurrent bonding lets a sponsor lock sentinel capital per unit of fee spent	Low	Acknowledged
I-01	Terminal arbitrated or timed out requests do not emit <code>OracleResult</code>	Informational	Acknowledged
I-02	Missing validation for non-zero <code>GOVERNANCE_DELAY</code> can disable governance timelocks	Informational	Fixed
I-03	Fee-reward division permanently locks undistributed remainders	Informational	Fixed
I-04	Non-revealer slashes are taken at freeze time and are never reversed	Informational	Fixed
I-05	<code>daoFeeShare</code> is never logged upon request creation	Informational	Fixed
I-06	Push-based settlement to sponsor allows freezing of requests and trapping of sentinel bonds	Informational	Fixed



[I-07](#)

Tokens with decimal precision larger than 18 may cause overflows

Informational

Fixed

Low Severity Issues

L-01`commit` accepts zero hashes, allowing repeated commits that lock bonds and distort accounting

Severity: **Low**

Impact: **Low**

Likelihood: **Low**

Files:

[/SentinelOracle.sol#L232-L238](#)

Status: Fixed

Description: The `checkNotCommitted` function determines existence by checking if `commitHash == 0`. However, a sentinel is permitted to submit `bytes32(0)` as their `commitHash`, bypassing the check on subsequent calls. This allows a sentinel to repeatedly commit to the same request, inflating committedCount and transferring multiple bonds. The duplicate bonds become permanently locked because the commitment record is overwritten, and the inflated count prevents the fast-finalization path from executing.

Recommendations: Reject `commitHash == bytes32(0)` in the `commit` function, and change the commitment existence check to use `vote != Vote.NONE`.

Customer's response: Fixed in commit [dab8ef9f](#)

Fix Review: Fixed

L-02 Unbounded concurrent bonding lets a sponsor lock sentinel capital per unit of fee spent, leading to sentinel exhaustion

Severity: **Low**

Impact: **Medium**

Likelihood: **Low**

Files:

[/SentinelOracle.sol#L232-L238](#)

Status: Acknowledged

Description: Every open request forces each participating sentinel to escrow terms `bondTarget = fee * bondMultiplier` from `commit` until it can claim after finalisation, and nothing bounds the number of requests that may be open simultaneously (`postRequest` is only gated on the trusted PROPOSER, which is permissionlessly reachable through `Consensus.proposeTransaction`).

A sponsor spending $k * \text{fee}$ therefore causes up to $k * \text{fee} * \text{bondMultiplier} * N$ of sentinel capital to be immobilised (N = number of active sentinels) for the whole commit+reveal window. Once a sentinel's fee-token balance/allowance is exhausted it can no longer commit at all; requests then reach the `nothingToReveal`/no-reveal branch of `finalize`, which refunds the sponsor's fee in full, so sustaining the attack after the exhaustion phase costs only gas. During that window legitimate proposals also time out, and because a terminal request permanently occupies its deterministic requestId, the affected Safe transactions cannot be re-proposed for the remainder of the epoch.

Recommendations: Bound the exposure, e.g. cap the number of simultaneously open requests, require the sponsor to escrow a non-refundable portion of the `fee` so that spam is not free on the timeout path

Customer's response: Acknowledged – “We do not plan to put any limitations on the proposal. It is not required by sentinels to participate, so the proposer risks having their fee locked up without a response.”

Fix Review: Acknowledged

Informational Issues

I-01. Terminal arbitrated or timed out requests do not emit `OracleResult`, stalling off-chain consumers

Description: The IOracle interface expects an `OracleResult` event to be emitted when a request reaches its terminal outcome, enabling off-chain clients to observe the result. While directly finalized requests emit this event, requests resolved via arbitration (`resolveDispute`) or timeouts (`finalize` timeout branch, `timeoutArbitration`, `markOutOfScope`) only emit custom events like `DisputeResolved` or `RequestTimedOut`. Any generic downstream consumer waiting for `OracleResult` will never observe the outcome of these requests and will stall indefinitely.

Recommendation: If necessary ensure that `OracleResult` is emitted in all terminal state transitions. Emit a positive or negative `OracleResult` alongside the specific dispute or timeout event when exiting via arbitration or timeout.

Customer's response: Acknowledged – *"This is expected behavior as the emission of OracleResult is a signal to the validators that there is a result. On arbitration and timeout we explicitly do not emit this result. For timeout this is obvious why it is done, for the arbitration case this is done because otherwise the arbitrator becomes an extreme point of failure (as they can set the final result and therefore mark a transaction as secure, not only insecure). Therefore for this first iteration a defensive approach was taken."*

I-02. Missing validation for non-zero `GOVERNANCE_DELAY` can disable governance timelocks

Description: The constructor validates other window variables but fails to check that `governanceDelay` is strictly greater than zero. If set to zero, all governance timelocks are disabled. Operations like adding sentinels or changing fee shares and receiver addresses will become effective in the exact same block, removing the intended delay-gating mechanism.

Recommendation: Ensure `params.governanceDelay > 0` is validated in the constructor.

Customer's response: Fixed in commit [ecea6a98](#)

Fix Review: Fixed by updating the comments on the assumptions behind the `governanceDelay` variable

I-03. Fee-reward division permanently locks undistributed remainders

Description: The reward for each winning sentinel is calculated as `fee / winningSideCount`. Any remainder from this integer division is never assigned or swept and becomes permanently trapped in the contract.

Recommendation: Allocate the remainder explicitly to the protocol or the final winning claimant.

Customer's response: Fixed in commit [e6866a3](#)

Fix Review: Fix confirmed. The remainder is correctly accounted for.

I-04. Non-revealer slashes are taken at freeze time and are never reversed, contradicting the documented "everyone made whole" behaviour of the timeout paths

Description: When `finalize` freezes a request it immediately slashes every committer that has not revealed:

JavaScript

```
uint128 nonRevealerCount = prog.committedCount - prog.revealedCount;  
unchecked { unrevealedBond = nonRevealerCount * self.terms.slashAmount; }
```

and `SentinelOracle.finalize` pushes that amount to the protocol funds receiver before the dispute has been adjudicated at all. If the dispute is later closed without any ruling (`timeoutArbitration` or `markOutOfScope` → `TIMED_OUT`), the slash is permanent: `slashAmountFor`'s `!isRevealedVote` branch still returns `terms.slashAmount` because `wasEstablished` is true, so the committer only ever recovers `bondTarget - slashAmount`.

This directly contradicts the invariant of the code documents for both timeout paths ("so bonds return in full via the existing `claim()/slashAmountFor` logic", "Same `TIMED_OUT` outcome ... no established side, every bond returns in full via `claim()`", and `SentinelOracle.timeoutArbitration`'s "every committed bond returns in full ... identical to the no-reveal timeout path"). An honest sentinel that suffers a transient RPC/host outage during the reveal window therefore loses `slashAmount` for a request whose outcome was never determined

Recommendation: Either defer the non-revealer slash until the dispute is actually adjudicated (accrue it in storage at `finalize` and only transfer it in `resolveDispute`), or explicitly document and reflect in the timeout paths that non-revealers are not made whole.

Refund the accrued unrevealed-bond slash on `timeoutArbitration/markOutOfScope` if the "no established outcome, everyone made whole" semantics are the intended ones.

Customer's response: Fixed in commit [4df721cbe](#)

Fix Review: Fixed confirmed

I-05. `daoFeeShare` is never logged upon request creation

Description: `create()` accepts `daoFeeShare` and stores it in `Terms`, but the event `NewRequest` omits it.

Since the DAO share is snapshotted per request and later determines each winner's payout (fee – `daoCut`), off-chain participants cannot reconstruct a request's economics from logs alone

Recommendation: Add `daoFeeShare` to `NewRequest` event

Customer's response: Fixed in commit [a56c62670](#)

Fix Review: Fixed confirmed

I-06. Push-based settlement to sponsor allows freezing of requests and trapping of sentinel bonds

Description: Several functions perform push-based token transfers to refund the sponsor address or pay the protocol funds receiver during state transitions. Because the sponsor address is arbitrary and controlled by the user, a malicious user can intentionally deploy a smart contract that reverts when it receives tokens that contain callbacks to the receiver, such as ERC777 tokens.

If a request is disputed or times out, the attempt to refund the sponsor reverts, permanently trapping the request in FROZEN or PENDING states. This prevents resolution and permanently traps the bonds of all honest sentinels who committed to the request.

Recommendation: Do not accept tokens with hooks as the fee token.

Customer's response: Fixed in commit [1d898012](#)

Fix Review: Fix confirmed

I-07. Tokens with decimal precision larger than 18 may cause overflows

Description: It should be noted that non-standard tokens with larger decimal precision may cause overflows of the `uint96 bondTarget` and `slashAmount` values. The maximum value for `uint96` is roughly `8E28`. Therefore, a token with ~24 decimal precision will hit overflow when multiplying it by `currentBondMultiplier` or `currentSlashingMultiplier`.

Recommendation: Ensure the use of standard 6 or 18 decimal ERC20 tokens.

Customer's response: Fixed in commit [1d898012](#)

Fix Review: Fix confirmed

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.